

UKIEPC 2025

Solutions

The Jury

18th October 2025

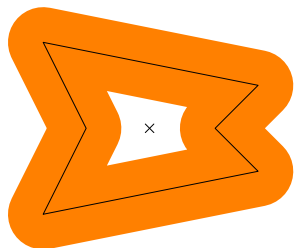
Arboreal Challenge

A polygon is burning from its edges. Find (one of) the last points to disappear.

Solution

- ▶ This is the same as finding **The largest inscribed circle** in the polygon.
- ▶ Although exact algorithms exist based on generalised Voronoi diagrams, they're difficult to code.
- ▶ What if we use a binary search? Say the answer is R :
 - ▶ Each edge creates an “exclusion zone” in the shape of a pill with radius R .
 - ▶ The pill is made up of a rectangle and two circular end-caps.
 - ▶ When the union of these zones fills the polygon, the fire is over.
 - ▶ When the union **does not** fill the polygon, we can find a solution at one of its vertices (a line-line, or line-circle, or circle-circle intersection).

Solution



- ▶ Solution: Binary search on R . At each pase, find every intersection of the exclusion zones.
- ▶ The closest point on the polygon needs to be checked. R away from all edges of the polygon, this is a possible answer.
- ▶ Keywords: *Binary search, Geometry, Inscribed circle*

Brickwork

Verify whether the union of a given set of rectangles itself forms a "wall" in the shape of a rectangle, with no overlaps (no point covered by two rectangles, except on borders).

Solution

- ▶ The wall needs to fulfil two conditions:
- ▶ **Area condition:** combined area must be equal to the area of the wall.
 - ▶ Check this by adding together all of the individual areas and comparing to the area of the bounding box of all rectangles.
- ▶ **Overlap condition:** None of the rectangles can overlap.
 - ▶ Just checking n^2 pair-wise intersections is too slow. Instead, use something like a sweep-line algorithm:
 - ▶ Go left-to-right and every time a rectangle *starts* or *ends*, add or remove its $(y, y + h)$ to a sorted set.
 - ▶ The set should never contain overlapping elements. You can check this by looking only at neighbours.
- ▶ Keywords: *Sweep*line, *Bounding* box

Colourful Captcha

Generate some ASCII art so that:

- ▶ it contains a given colour as a subword
- ▶ it does not contain other given colours as substrings
- ▶ it “reads” as another given colour topologically:
numbers of holes in letters should match

Solution

- ▶ Approaches for letter generation:
 - ▶ Lay out each possible letter to produce readable letters
 - ▶ In all letters in the given colours, there are either 0, 1 or 2 holes: generate only shapes with this number of holes
 - ▶ In all cases, pay attention to what we mean by holes
 - ▶ Ensure separation. Easiest: generate rectangles, ensure a horizontal interval of one whitespace

```
B . B B B . B B B
B . B . B . B . B
B . B . B . B B B
B . B . B . B . B
B . B B B . B B B
```

Depot

We have processes to convert x of item a into y of item b for various (x, a, y, b) . There is limited storage space in between. What is the maximum amount of the final item we can end up with?

Solution

- ▶ Each stage is an independent *knapsack problem* to maximise.
- ▶ The only thing that stops us using a greedy approach is that the space is limited.
- ▶ So we need to maximise $converted[x + y] = y$ without $x + y$ ever exceeding the limit.
- ▶ If $converted[x + y] = y$, then $converted[x + y - 1] = y - 1$ is also possible by discarding one of b .
- ▶ Run the normal knapsack DP algorithm, and then go back through the list making this adjustment.
- ▶ Keywords: *Dynamic programming, Knapsack*

Enclosure

Find the n -polygon with the largest area, given its perimeter as $d - n * m$. Too many vertices leads to a small polygon, and too few leads to inefficient coverage. A minimum of 3 vertices is required to enclose a non-zero area.

We have processes to convert x of item a into y of item b for various (x, a, y, b) . There is limited storage space in between. What is the maximum amount of the final item we can end up with?

Solution

- ▶ Observation 1: the polygon must be convex, since any 2 consecutive edges can be 'flipped' outwards to increase area without affecting the perimeter.
- ▶ Observation 2: the polygon must be regular. Suppose 2 neighbouring edges are unequal, we can move the shared vertex to maximize the area of the triangle formed by the 3 relevant vertices (the shared vertex traces out an ellipse). This occurs when the 2 edges are equal.
- ▶ The problem reduces to finding the best number of vertices, n , which can be achieved via a tertiary search. The complexity should be $O(t \log(d/m))$.
- ▶ Keywords: *Trigonometry*

Fell Walking

Find a path through a graph with weighted vertices $i \dots j$ such that $\max(w_i) - \min(w_i)$ is as small as possible.

Solution

- ▶ Assume we know $\min(w_i)$ and $\max(w_i)$. We can use any of the vertices in between.
- ▶ Create a graph g' which contains an edge if $\min(w) \leq (w_a, w_b) \leq \max(w)$.
- ▶ If this graph has a **spanning tree** containing vertices 1 and 2, it is a solution.
- ▶ Sort all of the edges by $\max(w_a, w_b)$.
- ▶ Go through every possible $\min(w_i)$ in order, and start adding allowed edges in order using *Kruskal's algorithm*.
- ▶ As long as the edges are sorted in advance, this costs $O(v \times e)$.
- ▶ Keywords: *Graphs, Minimum spanning tree, DSU*

Get Good

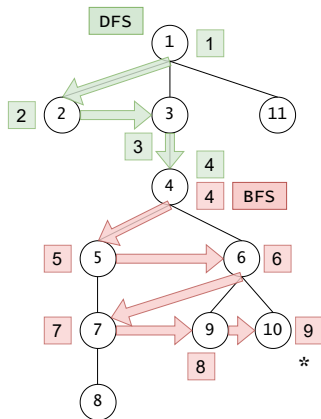
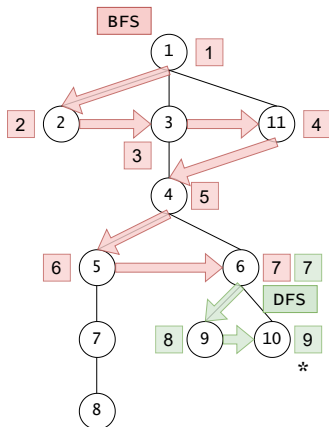
Problem summary: find maximum skill points through a combinations of work and rest.

Solution

- ▶ Observation 1: at the start, Charlie should simply work till tired, because $a \geq b$.
- ▶ Observation 2: from the tired state, Charlie has 2 options, rest y days and work x fresh days, or power through in a tired state.
- ▶ For integer multiples of $x+y$ days, we need to compare $x*a$ and $(x+y)*b$. For number of days less than $x+y$, we need to check as a special case.
- ▶ Keywords: *Greedy*,

Hybrid Search: Overview

- ▶ We have a tree with a fixed structure.
- ▶ We begin a BFS, or a DFS from the tree's root.
- ▶ We may change the type of search to the other one at most once.
- ▶ How quickly can we reach a node K?



Hybrid Search: Notations and a first observation

- ▶ The **expected complexity** is $O(N)$, but we also accept $O(N\sqrt{N})$.
- ▶ If we start from the root, let $\text{dft}[\text{node}]$ be the position in the DFS that we find node in ($\text{bft}[\text{node}]$ is the analog for BFS).
- ▶ Let $\text{ssz}[\text{node}]$ keep the subtree size of node.

Observation: Since we can only further visit the subtree of the node we switched in, we must switch in an ancestor of K (K inclusively, equivalent to not switching at all).

Hybrid Search: BFS to DFS

Property: the next $ssz[node] - 1$ nodes in the DFS after we visit *node* are exactly those from *node*'s subtrees.

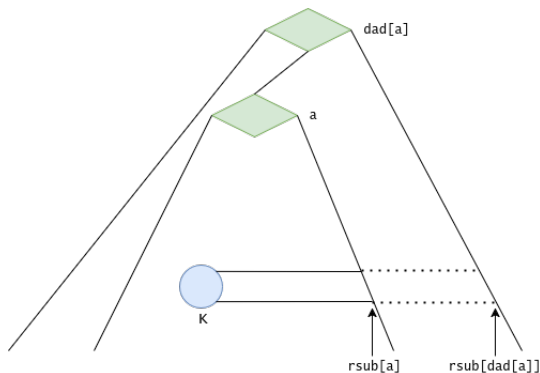
- ▶ As a result, if we switch to DFS in a , then we will visit K $dft[K] - dft[a]$ positions later.

Then, the minimum position for K in this case is:

$$\min(bft[a] + dft[K] - dft[a] \mid a \in \text{Ancestors}(K)) = \\ dft[K] + \min(bft[a] - dft[a] \mid a \in \text{Ancestors}(K))$$

Hybrid Search: DFS to BFS

Define (but don't compute for now) $rsub[a]$ as the number of nodes at the same level as K that are breadth-visited after K , and are in a 's subtree.



Hybrid Search: DFS to BFS

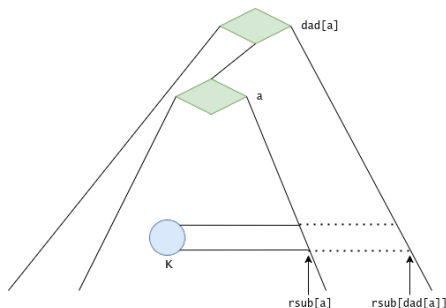
- Compute $ss'[node]$ as the subtree size of node, but count only nodes whose depth is at most equal to that of K .

Then the position for K if we switch in $a \in \text{Ancestors}(K)$ is:

$$dft[a] + ss'[a] - rsub[a]$$

since a 's subtree has $ss'[a]$ nodes down to K 's level inclusively, and exactly $rsub[a]$ of them will be visited after K by the BFS.

Hybrid Search: DFS to BFS - computing $rsub[]$

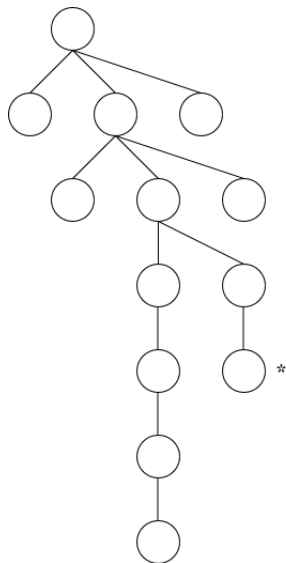


Property: the nodes found in $rsub[a]$ are also found in $rsub[dad[a]]$, along with some others immediately to their right.

- Stop advancing in the list by keeping time in, time out DFS counters for each tree node.

- We will iteratively move up the ancestor chain, starting from K and going up to the root.
- We can remember all of the nodes in the tree that are to the right of K , at the same depth as K in the normal BFS. We will update $rsub$ by adding from the list as we move towards the root on the ancestry chain. We need to stop when we are about to exit a 's subtree.

Hybrid Search: DFS to BFS - brute force



- ▶ What if we want to solve DFS to BFS by brute-force computing K 's BFS position in a 's subtree for all $a \in \text{Ancestors}(K)$?
- ▶ Can we be efficient by not computing for any node in $\text{Ancestors}(K)$? After all, it can have $\Theta(N)$ size.
- ▶ Generally, you can visually tell where it's best to change from DFS to BFS.
- ▶ Can you transform this visual cue into a generic property?

Hybrid Search: DFS to BFS - brute force

- ▶ If $x \in \text{Ancestors}$, $x \neq k$, then let:

$$\text{next}(A, x) = y \iff \text{dad}[y] = x, y \in A$$

where A is shorthand for Ancestors .

- ▶ Let the subtree additional size (only for nodes in A) be:

$$\text{ssa}[x] = \text{ssz}[x] - \text{ssz}[\text{next}(A, x)]$$

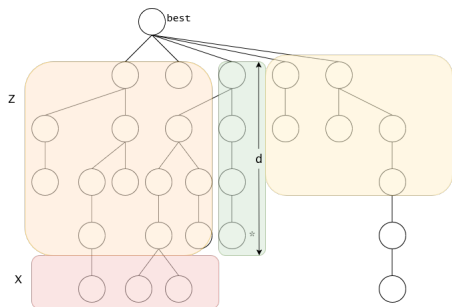
i.e. how many extra nodes we have added by going up the ancestor chain from $\text{next}(A, x)$ to x .

Hybrid Search: DFS to BFS - brute force

We claim that we can find the optimal result for DFS to BFS in $O(\sqrt{N})$ brute-force BFS runs by doing the following:

1. A DFS-only run, no change to BFS.
2. $\lceil \sqrt{N} \rceil$ brute-force runs from K 's closest ancestors.
3. $\lceil \sqrt{N} \rceil$ brute-force runs from the nodes $x \in A$ with the biggest size additions compared to their child in A , i.e. biggest $ssa[x]$ values.

Hybrid Search: DFS to BFS - brute force



- ▶ Let *best* be the optimal node in which to switch from DFS to BFS.
- ▶ Let *d* be the additional depth needed to reach *K*.
- ▶ Let *Z* be the set of nodes that will be visited regardless if we continue with DFS from *best* or change to BFS (excl. the *d*-length chain).
- ▶ *X* contains DFS-only, and *Y* BFS-only.
- ▶ Then $\text{ssz}[\text{best}] \geq 1 + d + |Z| + |X| + |Y|$.

Hybrid Search: DFS to BFS - brute force

- ▶ Suppose that in best we have $\text{length}(\text{DFS to } k) > \text{length}(\text{BFS to } k)$. If they are equal or smaller, then a DFS-only run would produce the optimal result, justifying the need for claim 1.
- ▶ $\text{length}(\text{DFS to } k) = |Z| + |X| + d + 1$,
 $\text{length}(\text{BFS to } k) = |Z| + |Y| + d + 1$, so we have $|X| > |Y|$, meaning that $|X| \geq 1$.
- ▶ Since $|X| \geq 1$, we must reach nodes in X , so Z has depth equal to d : $|Z| \geq d$.
- ▶ Since DFS produces a worse result, K cannot be found in the first child of best, otherwise switching in $\text{next}(A, \text{best})$ would be at least as good as switching in best (if we have equality, the proof implicitly shows that we will cover at least a best with the optimal value).

Hybrid Search: DFS to BFS - brute force

- ▶ Since K cannot be found in the first child of best and Z 's depth is d , then there are at least d nodes from Z that will disappear when we move down from best to $\text{next}(A, \text{best})$.
 - ▶ As a result, when we go up from $\text{next}(A, \text{best})$ up to best , we add at least $d + |X| + |Y| + 1 \geq d + 1 + 0 + 1 = d + 2$ nodes, so $\text{ssa}[\text{best}] \geq d + 2$.
 - ▶ **Observation:** $d \geq \sqrt{N}$, otherwise we would find the optimal answer in the runs from point 2 of the claim.
-
- ▶ We need at least $\lceil \sqrt{N} \rceil$ nodes in A for which $\text{ssa}[x] \geq \text{ssa}[\text{best}]$, such that best itself is omitted from point 3 runs.

Hybrid Search: DFS to BFS - brute force

- ▶ Since $ssa[]$ considers node additions, if $x, y \in A, x \neq y$, then nodes from $ssa[x]$ intersected with nodes from $ssa[y]$ is empty.

So the tree's number of nodes is at least:

$$\text{sum}(ssa[x] \mid ssa[x] \geq \text{KthBiggest}(ssa, \lceil \sqrt{N} \rceil)) \geq \dots$$

$$\dots \geq ssa[\text{best}] \cdot \lceil \sqrt{N} \rceil \geq (d+2) \cdot \lceil \sqrt{N} \rceil \geq (\sqrt{N} + 2) \cdot \sqrt{N} > N$$

Contradiction! So we can always cover at least a best which gives an optimal DFS to BFS answer with claims 1, 2, 3.

- ▶ Would ternary search on the Ancestors set work on finding best?
- ▶ Keywords: *Tree traversals, Tree properties.*

Itsy Bits

Find the smallest **power of two** number of bits that can hold an unsigned integer number.

Solution

- ▶ The input number always fits within 64 bits.
- ▶ There are only seven possible power-of-two answers:
1, 2, 4, 8, 16, 32, 64.
- ▶ Thus, just try them all out in order.
- ▶ It's a valid answer if $2^x > n$. But...
 - ▶ Make sure you use 64-bit types, and don't overflow with 2^{64} .
- ▶ Keywords: *Bits*

Joust Sort

Sort a string according to a (possibly inconsistent or underspecified) list of ad-hoc comparisons between letters.

Solution

- ▶ If $a < b$ and $b < c$, then clearly $a < c$.
- ▶ We can find the matrix of all of these implied conditions using the **Floyd-Warshall** all-pairs shortest path algorithm.
- ▶ If, for any two letters from the original string, we find that $a < b$ and $b < a$ (including the same letter), the answer is IMPOSSIBLE.
- ▶ Otherwise, sort the string either using the comparison matrix, or a standard topological sort algorithm, and print it.
- ▶ Keywords: *Topological sort*, *Floyd-Warshall*

Klaus

Arrange some triangles on three lines, such that none of them intersect each other.

Solution

- ▶ This is fundamentally an annoying dynamic programming task in 3 dimensions.
- ▶ Our DP function will place the next highest triangle on one of the 3 lines. The state will be the highest triangle placed on each line so far $dp[a][b][c]$.
- ▶ The next triangle's base must be higher than every one placed so far.
- ▶ Naively implementing this by searching for intersections yields an $O(N^4)$ algorithm, which can pass if optimized.
- ▶ However, you may keep an auxiliary table of running information from the lower DP states and use it to reduce runtime to $O(N^3)$, which passes easily.
- ▶ The main challenge is keeping the code simple.
- ▶ Keywords: *Dynamic programming*.

Last Orders

Visit pubs in any order, without overstaying your welcome or crawling to the same pub twice in a row.

Solution

- ▶ This is fundamentally a **non-annoying** dynamic programming task in **2** dimensions.
- ▶ We are going to optimize time taken to reach a given location and number of pints drunk so far.
- ▶ Thus our DP function will be `best_time(address, progress)`.
- ▶ We need to precalculate the distance between all of the pubs so that we know the fastest way to go from $best(x, p)$ to $best(y, p + 1)$ and immediately drink.
- ▶ We create a transition from (x, p) to $(y, p + 1)$ if x and y are different, and the time taken to reach and drink at pub y doesn't put us over its closing time.
- ▶ Keywords: *Dynamic programming*.

Motorway

Remove one item from a list so that $\max(\text{gap})$ is as small as possible.

Solution

- ▶ Just removing the items one-by-one is $O(N^2)$, which is too slow.
- ▶ Instead, record the **current** maximum gap once (this will not get smaller).
- ▶ And then, look for the smallest possible **new** gap we might create in $O(N)$.
- ▶ The answer is the maximum of both of them.
- ▶ Keywords: *Ad-hoc*.